

Deadlock: Dört Koşul ve Banker's Algorithm

İşletim Sistemleri — Konu 5 (Canlı Demo Materyali)

1. Deadlock Nedir?

Deadlock (kilitlenme), iki veya daha fazla sürecin, birbirlerinin elinde tuttuğu ve asla serbest bırakmayacağı kaynakları beklediği için hiçbirinin ilerleyemediği durumdur. Klasik örnek: Süreç A, kaynak X'i tutup kaynak Y'yi bekler; Süreç B, kaynak Y'yi tutup kaynak X'i bekler. İkisi de sonsuza kadar bekler.

2. Deadlock İçin Dört Gerekli Koşul (Coffman Koşulları)

Bir deadlock oluşabilmesi için aşağıdaki **dördü de aynı anda** sağlanmalıdır; bu koşullardan biri kırılırsa deadlock önlenmiş olur.

1. **Karşılıklı dışlama (mutual exclusion)**: En az bir kaynak paylaşılamaz biçimde (aynı anda yalnız bir süreç tarafından) tutulur.
2. **Tut ve bekle (hold and wait)**: Bir süreç en az bir kaynağı elinde tutarken, başka süreçlerin elinde olan ek kaynakları bekler.
3. **Önceliksiz alma (no preemption)**: Bir kaynak, onu tutan süreçten zorla alınmaz; yalnız süreç kendi isteğiyle bırakabilir.
4. **Dairesel bekleme (circular wait)**: Süreçler $\{P_0, P_1, \dots, P_n\}$ arasında, P_0 P_1 'in tuttuğu kaynağı bekler, P_1 P_2 'ninkini bekler, ..., P_n P_0 'inkini bekler biçiminde dairesel bir bekleme zinciri vardır.

3. Deadlock'la Başa Çıkma Stratejileri

- **Önleme (prevention)**: Dört koşuldan birini yapısal olarak imkansız kılmak. Örneğin tüm kaynakları tek seferde, işe başlamadan önce istetmek "tut ve bekle"yi ortadan kaldırır (ama kaynak kullanımını verimsizleştirir).
- **Kaçınma (avoidance)**: Sistemin, kaynak isteklerini yalnız güvenli bir duruma götürüyorsa onaylaması. **Banker's Algorithm** bu kategoridedir (aşağıda).
- **Tespit ve kurtarma (detection & recovery)**: Deadlock'un oluşmasına izin verip periyodik olarak kaynak ayırma grafiğinde döngü arayarak tespit etmek; bulunca bir süreci sonlandırıp veya kaynağını geri alıp kurtarmak.
- **Görmezden gelme (ostrich algorithm)**: Bazı genel amaçlı işletim sistemleri (ör. çoğu masaüstü Unix), deadlock nadir olduğu için önleme maliyetini göze almaz ve gerekirse yeniden başlatmaya güvenir.

4. Banker's Algorithm (Kaçınma)

Banker's Algorithm, her kaynak isteğini yalnızca sistemin **güvenli durumda (safe state)** kalmasını garanti ediyorsa onaylayan bir kaçınma algoritmasıdır. Bir durum güvenlidir, eğer tüm süreçlerin, mevcut kaynaklarla, bir sırayla (hepsi teker teker maksimum ihtiyaçlarını karşılayıp bitirilebilecek şekilde) tamamlanabileceği bir dizilim varsa.

Algoritma her süreç için şu bilgiyi tutar:

- Max: sürecin toplam maksimum kaynak ihtiyacı
- Allocation: şu an elinde tuttuğu kaynaklar
- Need = Max - Allocation: kalan potansiyel ihtiyaç

Bir istek geldiğinde algoritma, isteği **geçici olarak** onaylayıp sistemin hâlâ güvenli bir durumda olup olmadığını kontrol eden bir **güvenlik algoritması (safety algorithm)** çalıştırır; güvenli değilse istek reddedilip süreç bekletilir (isteğin kendisi uygulanmaz). n süreç ve m kaynak türü için güvenlik algoritmasının maliyeti $O(m \times n^2)$ mertebesindedir.

Sınırlılık: Banker's Algorithm, her sürecin maksimum kaynak ihtiyacını **önceden** bilmeyi gerektirir; bu, pratikte çoğu genel amaçlı sistemde gerçekçi değildir. Bu yüzden gerçek işletim sistemlerinde tespit-ve-kurtarma veya basitçe göz ardı etme daha yaygındır.

5. Kaynak Ayırma Grafiği ile Tespit

Kaynak ayırma grafiğinde (resource allocation graph) süreçler ve kaynaklar düğüm, "tutuyor" ve "bekliyor" ilişkileri kenardır. Her kaynak türünden yalnız bir örnek varsa, grafikte bir **döngü** varlığı deadlock'un **gerekli ve yeterli** koşuludur. Birden fazla örnekli kaynak türlerinde döngü varlığı gerekli ama yeterli değildir; tam tespit için Banker's Algorithm'e benzer bir azaltma (reduction) prosedürü gerekir.

6. Özet

- Deadlock, dört koşulun (karşılıklı dışlama, tut-ve-bekle, önceliksiz alma, dairesel bekleme) aynı anda sağlanmasıyla oluşur; birini kırmak deadlock'u önler.
- Banker's Algorithm, her isteği yalnız sistemi güvenli durumda tutuyorsa onaylayarak deadlock'u **önceden kaçınma** ile engeller; maliyeti $O(m \times n^2)$ 'dir ve önceden bilinen maksimum ihtiyaç varsayımına dayanır.
- Alternatif stratejiler: önleme (koşulu yapısal olarak imkansız kılmak), tespit + kurtarma (döngü arama), ve göz ardı etme.